

Joint modeling of longitudinal and survival data

Efficient Bayesian Analysis of Longitudinal and Survival Data Using R-INLA
ISCB 2026, Freiburg, Germany

Denis Rustand

September 27, 2026

```
library(INLA)
library(INLAjoint)
library(survival)
library(ggplot2)
library(ggpubr)
library(mvtnorm)
library(PermAlgo)
```

Let us consider a mixed-effects model for longitudinal data including fixed effects β , a random intercept b_{i0} and a random slope of time b_{i1} :

$$E[Y_i(t)] = \eta_i(t) = \beta_0 + b_{i0} + (\beta_1 + b_{i1}) \cdot t + \beta_2 \cdot trt_i + \beta_3 \cdot trt_i \cdot t$$

We can assume some relationship between the longitudinal and a survival model based on some components of $\eta_i(t)$. The risk of event is defined as:

$$\lambda_i(t) = \lambda_0(t) \exp(h(\eta_i(t)))$$

where $h(\eta_i(t))$ is a function of the parameters of the longitudinal model. Several association structures have been proposed in the literature, each one having a specific interpretation and purpose.

Let's simulate the latent variables and the longitudinal observations that will be used across all examples of this session:

```
set.seed(1)
n <- 1000 # number of individuals
fw <- 5 # follow-up duration
b_0 <- 1 # intercept
b_1 <- -1 # slope
b_2 <- 1 # trt
b_3 <- 0.5 # trt x slope
b_e <- 0.5 # residual error

mestime <- seq(0, fw, 1) # measurement times
n_i <- length(mestime)
time <- rep(mestime, n) # time column
id <- rep(1:n, each = n_i) # individual id

b1SD <- 1 # random intercept SD
b2SD <- 0.7 # random slope SD
```

```

cor_bintslo <- 0.6 # correlation
cov_bintslo <- b1SD * b2SD * cor_bintslo
Sigma <- matrix(c(b1SD^2, cov_bintslo,
                 cov_bintslo, b2SD^2), ncol = 2)

MVnorm <- rmvnorm(n, rep(0, 2), Sigma)
b1 <- rep(MVnorm[, 1], each = n_i) # random intercepts
b2 <- rep(MVnorm[, 2], each = n_i) # random slopes

trt <- rbinom(n, 1, 0.5)
trtY <- rep(trt, each = length(mestime))

linPred <- b_0 + b1 + (b_1 + b2) * time + b_2 * trtY + b_3 * trtY * time
Y <- rnorm(n_i * n, linPred, b_e)
datJM_Lt <- data.frame(id, time, Y, trt = trtY)

```

Plot setup: posterior marginals with true values

We pre-define the true parameter values and reusable plot code strings. For each fitted model we only need to update `Plots_jm` and `TV3` (the true association values). The dashed red lines indicate the true values used for data generation.

```

# True values shared by all simulation models
TV1 <- c(b_0, b_1, b_2, b_3, b_e^2) # fixed effects + residual variance
TV2 <- c(Sigma) # variance-covariance matrix elements

# Plot code strings (evaluated after updating Plots_jm and TV3)
P1 <- "Plots_jm$Outcomes$L1 +
  geom_vline(aes(xintercept = TrueValues),
             data = data.frame(Effect = unique(Plots_jm$Outcomes$L1$data$Effect),
                               TrueValues = TV1),
             colour = 2, linetype = 2) +
  theme_minimal() +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())"

P2 <- "Plots_jm$Covariances$L1 +
  geom_vline(aes(xintercept = TrueValues),
             data = data.frame(Effect = unique(Plots_jm$Covariances$L1$data$Effect),
                               TrueValues = TV2),
             colour = 2, linetype = 2) +
  theme_minimal() +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank(),
        legend.position = 'none')"

P3 <- "Plots_jm$Associations +
  geom_vline(aes(xintercept = TrueValues),
             data = data.frame(Effect = unique(Plots_jm$Associations$data$Effect),
                               TrueValues = TV3),
             colour = 2, linetype = 2) +
  theme_minimal() +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())"

```

Shared random effects independent (SRE_ind)

The **shared random effects independent** structure links the longitudinal and survival processes by sharing each random effect separately. An individual with similar deviations from the average longitudinal profile is assumed to have a similar survival outcome. This structure is computationally simple because the shared components are time-constant:

$$\lambda_i(t) = \lambda_0(t) \exp(\alpha_0 \cdot b_{i0} + \alpha_1 \cdot b_{i1})$$

The fixed effects α_0 and α_1 scale each random effect independently in the survival submodel. A positive α_0 means that individuals whose baseline marker level exceeds the population average have an elevated hazard at all time points, similarly for α_1 and the slope. Because the shared components are time-fixed, no numerical integration over time is needed, making this the most computationally efficient association structure.

```
baseScale <- 0.1 # baseline hazard scale (exponential)
alpha_0 <- 0.5 # first association parameter
alpha_1 <- 1 # second association parameter
u <- runif(n)
eventTimes <- -(log(u) / (baseScale * exp(MVnorm[, 1] * alpha_0 + MVnorm[, 2] * alpha_1)))
eventIndic <- as.numeric(eventTimes < fw)
eventTimes <- pmin(eventTimes, fw)

datJM_S <- data.frame(id = 1:n, eventTimes, eventIndic, trt)

# Remove longitudinal measurements recorded after the event
datJM_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which(datJM_Lt$time >= datJM_S$eventTimes[datJM_S$id == x] &
    datJM_Lt$id == x))), ]

cat("Event rate:", round(mean(eventIndic) * 100, 1), "%\n")
```

```
## Event rate: 43.5 %
```

```
cat("Median follow-up:", round(median(eventTimes), 2), "years\n")
```

```
## Median follow-up: 5 years
```

```
head(datJM_S)
```

```
##   id eventTimes eventIndic trt
## 1  1  2.423559         1    0
## 2  2  5.000000         0    1
## 3  3  5.000000         0    0
## 4  4  0.209069         1    0
## 5  5  4.201830         1    1
## 6  6  1.193374         1    0
```

```
head(datJM_L)
```

```
##   id time          Y trt
## 1  1  0 -0.4599131  0
## 2  1  1 -0.9396970  0
## 3  1  2 -1.8808611  0
## 7  2  0  0.9412211  1
## 8  2  1  1.4645422  1
## 9  2  2  1.7097579  1
```

We fit the joint model using `assoc = "SRE_ind"`:

```

M_jm_sre_ind <- joint(
  formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong = Y ~ time * trt + (1 + time | id),
  dataSurv = datJM_S, dataLong = datJM_L,
  id = "id", timeVar = "time", assoc = "SRE_ind",
  control = list(int.strategy = "eb")
)

summary(M_jm_sre_ind, sdcor = TRUE)

## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    0.9311 0.0473    0.8384    0.9311    1.0238
## time_L1         -0.9876 0.0339   -1.0540   -0.9876   -0.9212
## trt_L1           1.0617 0.0617    0.9408    1.0617    1.1826
## time:trt_L1      0.4920 0.0430    0.4078    0.4920    0.5762
## Res. err. (sd)  0.5144 0.0079    0.4992    0.5144    0.5300
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1      1.0437 0.0285    0.9882    1.0432    1.0998
## time_L1           0.7201 0.0223    0.6768    0.7194    0.7653
## Intercept_L1:time_L1 0.6383 0.0281    0.5809    0.6394    0.6897
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.0508 0.0521    0.0035    0.0347    0.1946
##
## Association longitudinal - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## SRE_Intercept_L1_S1 0.5543 0.0901    0.3777    0.5540    0.7325
## SRE_time_L1_S1      1.0265 0.1500    0.7302    1.0269    1.3208
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -7446.916                      -7440.195
##
## Deviance Information Criterion: 11447.89
## Widely applicable Bayesian information criterion: 11482.13
## Computation time: 8.63 seconds

```

Hazard ratio interpretation. We can convert the association parameters to hazard ratios:

```

round(summary(M_jm_sre_ind, hr = TRUE)$AssocLS, 3)

##               exp(mean)      sd 0.025quant 0.5quant 0.975quant
## SRE_Intercept_L1_S1    1.748 0.155    1.463    1.740    2.074
## SRE_time_L1_S1         2.822 0.418    2.085    2.792    3.727

```

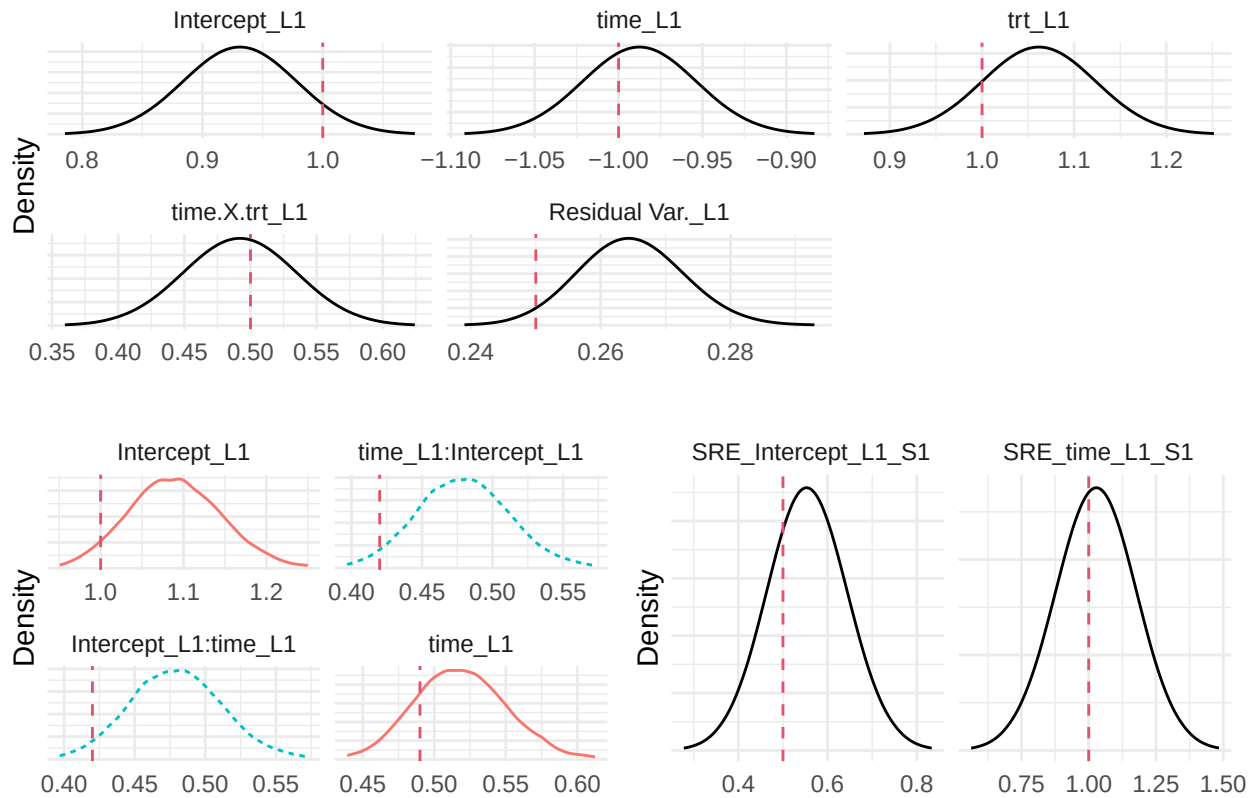
The model recovers the true values ($\alpha_0 = 0.5$, $\alpha_1 = 1$). Both association parameters are positive: an individual whose baseline marker level is 1 unit above the population average faces a hazard 75% higher at any given time. The posterior marginal distributions confirm recovery of all model parameters (dashed red lines indicate true values):

```

Plots_jm <- plot(M_jm_sre_ind)
TV3 <- c(alpha_0, alpha_1)
ggarrange(eval(parse(text = P1)),

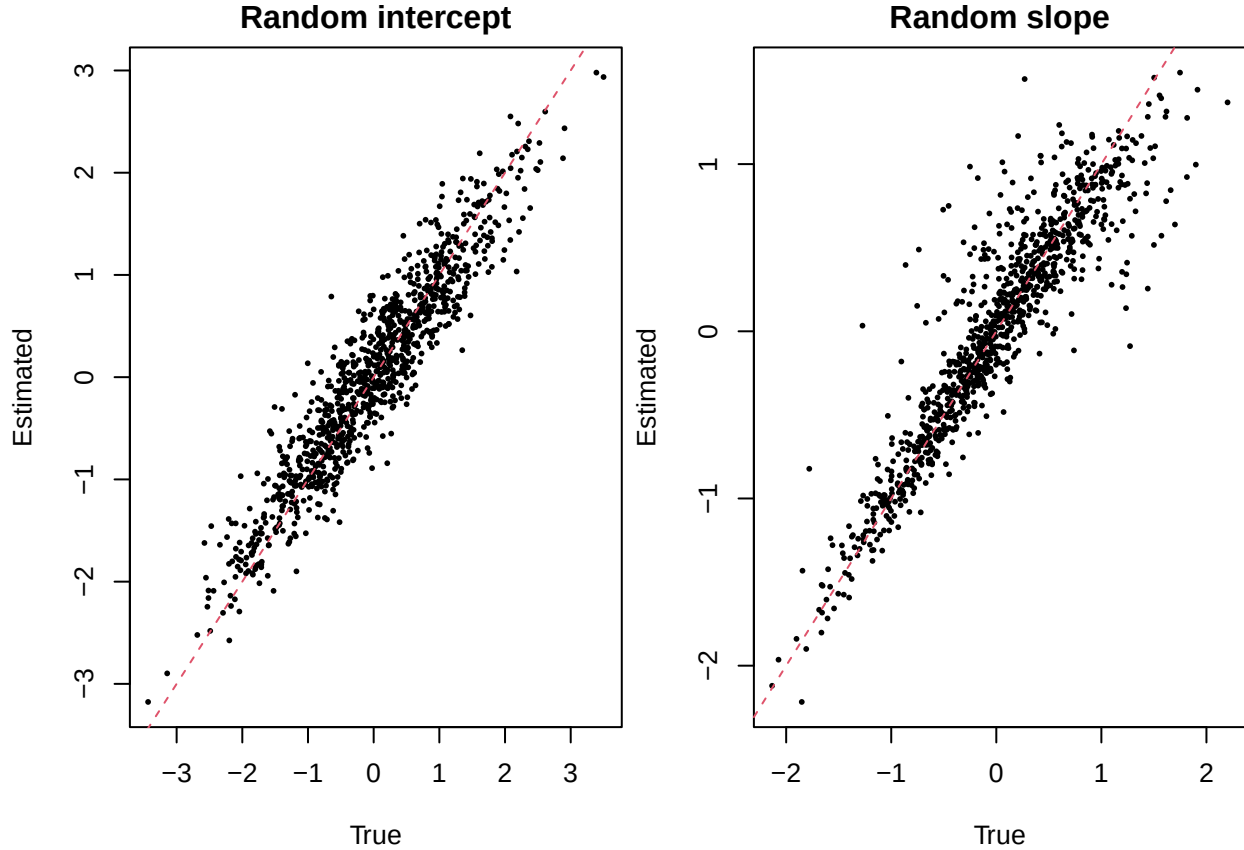
```

```
ggarrange(eval(parse(text = P2)),
           eval(parse(text = P3)), ncol = 2), nrow = 2)
```



We can also verify that the model recovered the individual random effects:

```
par(mfrow = c(1, 2), mar = c(4, 4, 2, 0.3), cex = 0.8)
plot(MVnorm[, 1], M_jm_sre_ind$summary.random$IDIntercept_L1$mean[1:n],
     xlab = "True", ylab = "Estimated", main = "Random intercept", pch = 16, cex = 0.5)
abline(0, 1, col = 2, lty = 2)
plot(MVnorm[, 2], M_jm_sre_ind$summary.random$IDtime_L1$mean[-(1:n)],
     xlab = "True", ylab = "Estimated", main = "Random slope", pch = 16, cex = 0.5)
abline(0, 1, col = 2, lty = 2)
```



Time-dependent individual deviation from the average longitudinal trend (SRE)

Building upon the previous structure, the **time-dependent shared random effects** association allows the shared component to evolve over time. Instead of sharing each random effect independently, the model shares their linear combination evaluated at the current time t :

$$\lambda_i(t) = \lambda_0(t) \exp(\alpha \cdot (b_{i0} + b_{i1} \cdot t))$$

This captures the *dynamic* nature of the individual's deviation from the population trajectory. Consider an individual with a high positive random intercept ($b_{i0} > 0$) but a strong negative random slope ($b_{i1} < 0$). At early time points their deviation from the mean is positive, so $\alpha > 0$ implies elevated hazard. As time progresses their trajectory crosses below the population average, so the term $(b_{i0} + b_{i1}t)$ becomes negative, leading to decreased hazard compared to the population. This structure captures how an individual's risk profile evolves as their biomarker deviates from the average over follow-up.

Because the hazard now includes a time-dependent component, inverse transform sampling no longer has a closed form. We use the `PermAlgo` package to simulate event times conditional on this time-varying covariate:

```
set.seed(2)
alpha <- 0.5
gap2 <- 0.01
mestime2 <- seq(0, fw, gap2)
time2 <- rep(mestime2, n)
n_i2 <- length(mestime2)
trt2 <- rep(trt, each = n_i2)
b12 <- rep(MVnorm[, 1], each = n_i2)
```

```

b22 <- rep(MVnorm[, 2], each = n_i2)

SRE_i <- b12 + b22 * time2 # individual deviation at time t

DatTmp <- permalgorithm(n, n_i2, Xmat = matrix(SRE_i, ncol = 1), betas = alpha)
DatTmp2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = TRUE)),
                  c("Id", "Event", "Stop")]

datJM2_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop + 1],
                      eventIndic = DatTmp2$Event, trt = trt)

datJM2_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which(datJM_Lt$time >= datJM2_S$eventTimes[datJM2_S$id == x] &
    datJM_Lt$id == x))), ]

```

We first set up the model without running the algorithm (`run = FALSE`) to inspect the internal data structure that INLAjoint constructs to evaluate the time-dependent association:

```

M_jmSRE <- joint(
  formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong = Y ~ time * trt + (1 + time | id),
  dataSurv = datJM2_S, dataLong = datJM2_L,
  id = "id", timeVar = "time", assoc = "SRE",
  control = list(int.strategy = "eb"), run = FALSE
)

```

We can inspect the internal data structure for the first individual. The longitudinal components:

```

id1 <- which(M_jmSRE$.args$data$IDIntercept_L1 == 1 |
             M_jmSRE$.args$data$expand1..coxph == 1)
sapply(M_jmSRE$.args$data[c(1:6, 8, 11)], function(x) round(x[id1], 2))

```

##	Intercept_L1	time_L1	trt_L1	time.X.trt_L1	IDIntercept_L1	IDtime_L1	Wtime_L1	Y_L1
## [1,]	1	0	0	0	1	1001	0.00	-0.46
## [2,]	1	1	0	0	1	1001	1.00	-0.94
## [3,]	NA	NA	NA	NA	1	1001	0.16	NA
## [4,]	NA	NA	NA	NA	1	1001	0.49	NA
## [5,]	NA	NA	NA	NA	1	1001	0.82	NA
## [6,]	NA	NA	NA	NA	1	1001	1.15	NA
## [7,]	NA	NA	NA	NA	1	1001	1.48	NA
## [8,]	NA	NA	NA	NA	1	1001	1.81	NA
## [9,]	NA	NA	NA	NA	NA	NA	NA	NA
## [10,]	NA	NA	NA	NA	NA	NA	NA	NA
## [11,]	NA	NA	NA	NA	NA	NA	NA	NA
## [12,]	NA	NA	NA	NA	NA	NA	NA	NA
## [13,]	NA	NA	NA	NA	NA	NA	NA	NA
## [14,]	NA	NA	NA	NA	NA	NA	NA	NA

The first two rows belong to the longitudinal submodel. The next six rows are used to compute the linear combination of random effects at the midpoint of each survival interval. And the survival components:

```

sapply(M_jmSRE$.args$data[c(14, 16, 8, 9, 21, 13)], function(x) round(x[id1], 2))

```

##	E..coxph	baseline1.hazard	Wtime_L1	usre1	SRE_L1_S1	y1..coxph
## [1,]	NA	NA	0.00	NA	NA	NA
## [2,]	NA	NA	1.00	NA	NA	NA

```
## [3,]      NA      NA      0.16      1      NA      NA
## [4,]      NA      NA      0.49      2      NA      NA
## [5,]      NA      NA      0.82      3      NA      NA
## [6,]      NA      NA      1.15      4      NA      NA
## [7,]      NA      NA      1.48      5      NA      NA
## [8,]      NA      NA      1.81      6      NA      NA
## [9,]    0.33      0.00      NA      NA      1      0
## [10,]   0.33      0.33      NA      NA      2      0
## [11,]   0.33      0.66      NA      NA      3      0
## [12,]   0.33      0.99      NA      NA      4      0
## [13,]   0.33      1.31      NA      NA      5      0
## [14,]   0.19      1.64      NA      NA      6      1
```

The column `SRE_L1_S1` links the computed linear combination to the survival submodel. Row 3 shows the combination is evaluated at time ≈ 0.16 , the midpoint of the first survival interval $[0, 0.33]$. More intervals gives a finer representation of the evolving association in the risk function.

We now run the model:

```
M_jmSRE <- joint.run(M_jmSRE)
summary(M_jmSRE, sdcor = TRUE)
```

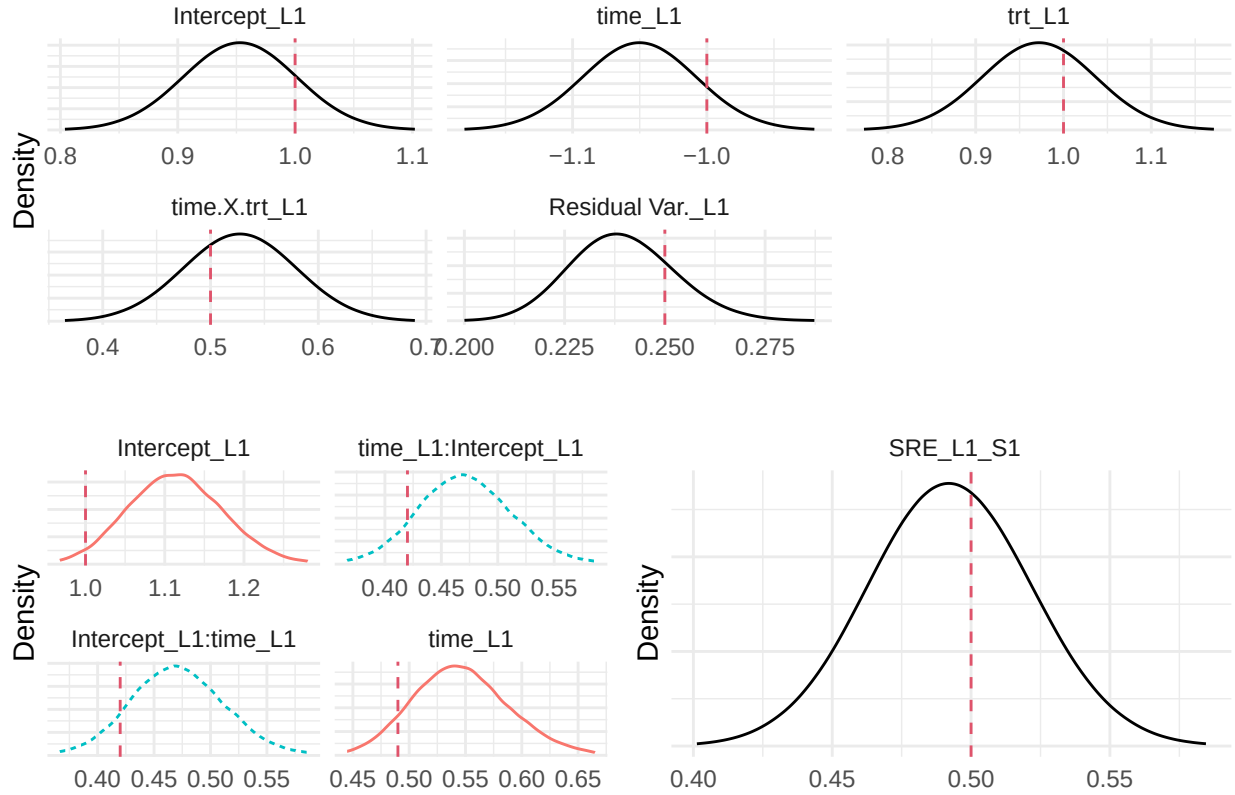
```
## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    0.9529 0.0484    0.8580    0.9529    1.0479
## time_L1        -1.0502 0.0423   -1.1332   -1.0502   -0.9672
## trt_L1          0.9720 0.0647    0.8451    0.9720    1.0989
## time:trt_L1     0.5272 0.0527    0.4239    0.5272    0.6306
## Res. err. (sd)  0.4891 0.0133    0.4638    0.4889    0.5161
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    1.0554 0.0292    0.9997    1.0553    1.1136
## time_L1         0.7383 0.0281    0.6850    0.7377    0.7935
## Intercept_L1:time_L1 0.6065 0.0435    0.5137    0.6079    0.6852
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.7526 0.1333    0.5258    0.7407    1.0487
##
## Association longitudinal - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## SRE_L1_S1 0.4925 0.0297    0.4342    0.4924    0.5513
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -27151.64                      -27145.84
##
## Deviance Information Criterion: 8239.127
## Widely applicable Bayesian information criterion: 8147.772
## Computation time: 10 seconds
```

The single coefficient `SRE_L1_S1` quantifies the association between the subject's current deviation from the population-average trajectory and the risk of event. The posterior marginals confirm recovery of all parameters:

```

Plots_jm <- plot(M_jmSRE)
TV3 <- c(alpha)
ggarrange(eval(parse(text = P1)),
           ggarrange(eval(parse(text = P2)),
                     eval(parse(text = P3)), ncol = 2), nrow = 2)

```



Current value (CV)

The **current value** association assumes the risk of the event at time t depends on the *true* expected value of the entire linear predictor from the longitudinal submodel, sometimes called the true underlying value of the biomarker since it excludes the measurement error ε :

$$\lambda_i(t) = \lambda_0(t) \exp(\alpha \cdot \eta_i(t))$$

Unlike the SRE structure, the CV structure incorporates the influence of **all covariates** (including treatment effects) on the longitudinal marker directly into the risk calculation. The association parameter α is interpreted as the log-hazard ratio for a one-unit increase in the true biomarker value. Of note, the CV and SRE associations are equivalent when there are no covariates in the longitudinal model (or when those covariates are also included in the survival part), because in those cases the hazard is shifted by a constant.

```

set.seed(2)
alpha <- 0.5
CV_i <- b_0 + b12 + (b_1 + b22) * time2 + b_2 * trt2 + b_3 * trt2 * time2

DatTmp <- permalgorithm(n, n_i2, Xmat = matrix(CV_i, ncol = 1), betas = c(alpha))
DatTmp2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = TRUE)),

```

```

      c("Id", "Event", "Stop")]

datJM3_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop + 1],
                      eventIndic = DatTmp2$Event, trt = trt)

datJM3_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which(datJM_Lt$time >= datJM3_S$eventTimes[datJM3_S$id == x] &
    datJM_Lt$id == x))), ]

```

We fit the model with `assoc = "CV"`:

```

M_jmCV <- joint(
  formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong  = Y ~ time * trt + (1 + time | id),
  dataSurv  = datJM3_S, dataLong = datJM3_L,
  id = "id", timeVar = "time", assoc = "CV",
  control = list(int.strategy = "eb")
)

summary(M_jmCV, sdcor = TRUE)

## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    0.9801 0.0511    0.8800    0.9801    1.0802
## time_L1        -1.0298 0.0443   -1.1165   -1.0298   -0.9430
## trt_L1          0.9648 0.0722    0.8233    0.9648    1.1064
## time:trt_L1     0.5643 0.0662    0.4346    0.5643    0.6939
## Res. err. (sd)  0.4992 0.0136    0.4730    0.4990    0.5266
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    1.0445 0.0295    0.9894    1.0434    1.1051
## time_L1         0.7240 0.0290    0.6675    0.7237    0.7822
## Intercept_L1:time_L1 0.6239 0.0428    0.5384    0.6259    0.7036
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.9155 0.1555    0.6231    0.9141    1.2283
##
## Association longitudinal - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## CV_L1_S1 0.5208 0.0295    0.463    0.5208    0.5792
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -27142.24                      -27136.44
##
## Deviance Information Criterion: 8245.696
## Widely applicable Bayesian information criterion: 8158.228
## Computation time: 9.26 seconds

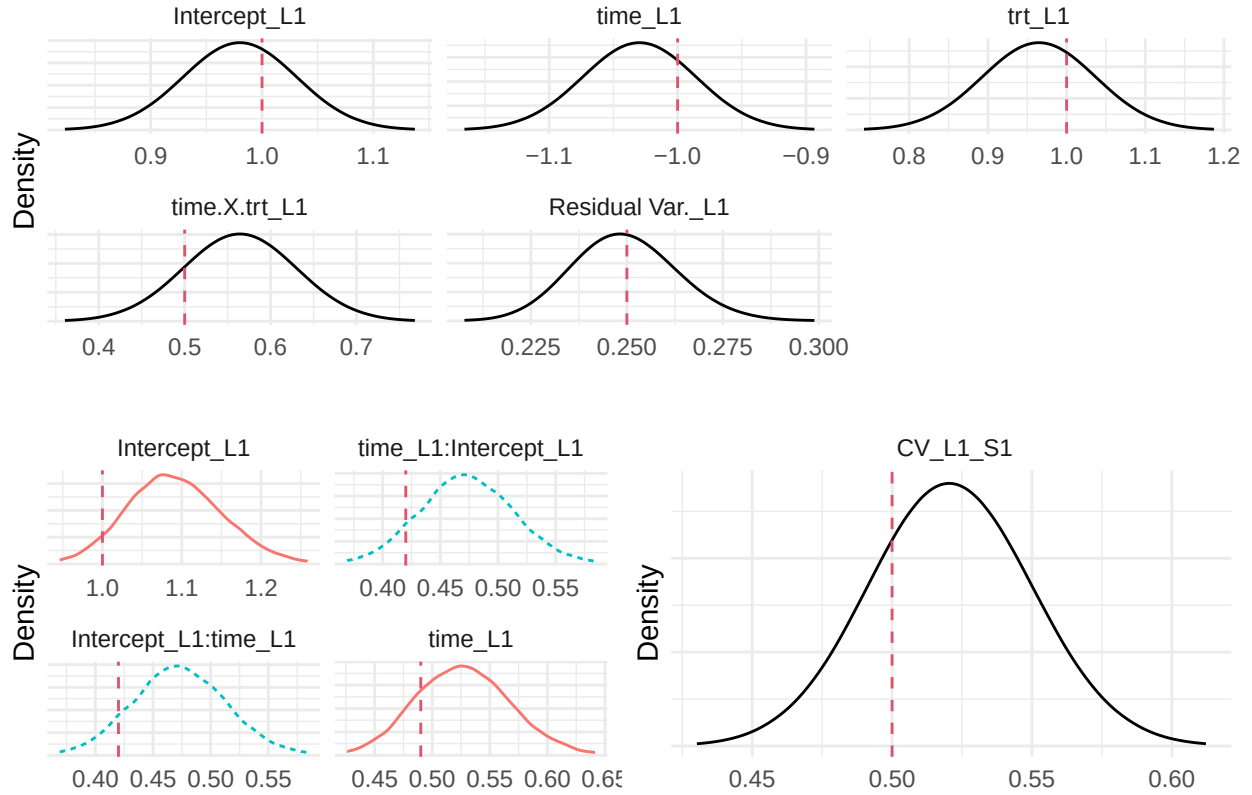
```

Interpretation. The association parameter `CV_L1_S1` represents the log-hazard ratio per one-unit increase in the true marker value at any time t . A positive value means that individuals whose true marker value is currently high face an elevated risk of the event at that moment. Because treatment affects the longitudinal trajectory (through β_2 and β_3), treatment's effect on survival operates indirectly through this association, a key advantage of the joint model.

```

Plots_jm <- plot(M_jmCV)
TV3 <- c(alpha)
ggarrange(eval(parse(text = P1)),
           ggarrange(eval(parse(text = P2)),
                     eval(parse(text = P3))), ncol = 2), nrow = 2)

```



Current slope (CS)

The **current slope** association focuses on the *dynamic behaviour* of the marker rather than its level. It uses the derivative of the linear predictor with respect to time, which describes the rate of change at time t :

$$\lambda_i(t) = \lambda_0(t) \exp\left(\alpha \cdot \frac{d\eta_i(t)}{dt}\right)$$

For our random intercept-slope model, the derivative is simply $\beta_1 + b_{i1} + \beta_3 \cdot trt_i$, which is the individual-specific slope. This structure can answer a distinct clinical question: **do two patients with different rates of change have different risks?** For instance, two patients both at biomarker value 5, one whose marker is rapidly increasing, one whose marker is stable, have the same risk under the CV model but potentially very different risks under the CS model. A positive α means that a more rapidly increasing marker at time t translates to a higher instantaneous hazard, independently of the current level. Both CV and CS can be considered simultaneously with “CV_CS”.

For more complex longitudinal models, `INLAjoint` computes the derivative numerically via the `numDeriv` package.

```

set.seed(4)
alpha <- 1
CS_i <- (b_1 + b22) + b_3 * trt2  # slope of the linear predictor

DatTmp <- permalgorithm(n, n_i2, Xmat = matrix(CS_i, ncol = 1), betas = c(alpha))
DatTmp2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = TRUE)),
                  c("Id", "Event", "Stop")]

datJM4_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop + 1],
                      eventIndic = DatTmp2$Event, trt = trt)

datJM4_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which(datJM_Lt$time >= datJM4_S$eventTimes[datJM4_S$id == x] &
    datJM_Lt$id == x))), ]

```

We fit the model with `assoc = "CS"`:

```

M_jmCS <- joint(
  formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong = Y ~ time * trt + (1 + time | id),
  dataSurv = datJM4_S, dataLong = datJM4_L,
  id = "id", timeVar = "time", assoc = "CS",
  control = list(int.strategy = "eb")
)

summary(M_jmCS, sdcor = TRUE)

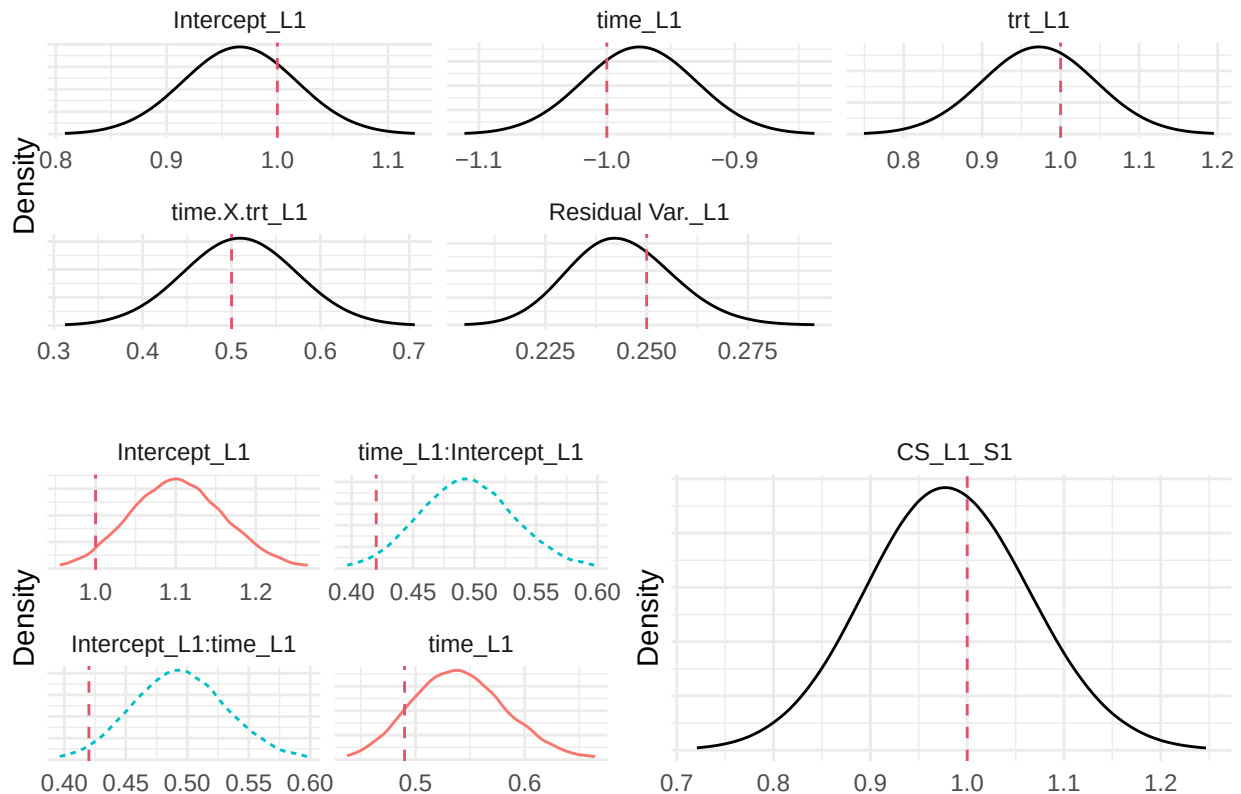
## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    0.9662 0.0513    0.8656    0.9662    1.0668
## time_L1        -0.9745 0.0444   -1.0616   -0.9745   -0.8875
## trt_L1          0.9724 0.0724    0.8304    0.9724    1.1143
## time:trt_L1     0.5094 0.0640    0.3840    0.5094    0.6348
## Res. err. (sd)  0.4936 0.0130    0.4689    0.4933    0.5201
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    1.0494 0.0277    0.9956    1.0490    1.1048
## time_L1          0.7347 0.0303    0.6779    0.7339    0.7950
## Intercept_L1:time_L1 0.6402 0.0393    0.5602    0.6426    0.7107
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.4662 0.1118    0.2868    0.4526    0.7245
##
## Association longitudinal - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## CS_L1_S1 0.9801 0.0853    0.8137    0.9796    1.1496
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -27629.58                      -27623.78
##
## Deviance Information Criterion: 8502.885
## Widely applicable Bayesian information criterion: 8440.118

```

```
## Computation time: 9.99 seconds
```

Interpretation. The parameter `CS_L1_S1` is the log-hazard ratio for a one-unit increase in the instantaneous rate of change. Because the slope is time-constant in our linear model, this structure induces a time-constant effect here. In a more realistic setting with non-linear longitudinal trajectories (e.g., initial decline followed by plateau), the derivative varies over time and the CS structure captures the prognostic value of the trend at each moment independently of the current level.

```
Plots_jm <- plot(M_jmCS)
TV3 <- c(alpha)
ggarrange(eval(parse(text = P1)),
           ggarrange(eval(parse(text = P2)),
                     eval(parse(text = P3)), ncol = 2), nrow = 2)
```



Current slope with non-linear trends

In the previous example the longitudinal model was strictly linear. The current slope was simply the individual rate of change which remained constant over time. Real biomarker data often show more complex dynamics. As we saw in the longitudinal session we can model non-linear trends by adding functions of time like quadratic terms or splines.

When we introduce a non-linear term the current slope becomes time-dependent. The rate of change evolves at each moment. To link this changing slope to the survival risk `INLAjoint` needs to compute the derivative of the linear predictor.

To do this the software uses `numDeriv` which is the standard package in R for numerical derivatives. You only need to define your time function in R and pass it to the model formula. `INLAjoint` will automatically evaluate the numerical derivative to compute the current slope at any given time. This makes it possible to

include any function of time easily.

Let us update our simulation to include a quadratic term. We define a simple function `f1 <- function(x) x^2` and add it to the fixed and random effects. The survival risk is now driven by the time-varying derivative of this quadratic profile.

```
set.seed(5)
alpha <- 1
f1 <- function(x) x^2
n <- 500
mestime <- seq(0, fw, 0.1) # more repeated measurements (every 0.1 units of time)
n_i <- length(mestime)
time <- rep(mestime, n)
id <- rep(1:n, each = n_i)
trt <- rbinom(n, 1, 0.5)
trtY <- rep(trt, each = length(mestime))
sds <- c(0.5, 0.5, 0.7)
cor_val <- 0.3
R <- matrix(cor_val, 3, 3); diag(R) <- 1
Sigma <- diag(sds) %*% R %*% diag(sds)
MVnorm <- rmvnorm(n, mean = rep(0, 3), sigma = Sigma)
b1 <- rep(MVnorm[, 1], each = n_i)
b2 <- rep(MVnorm[, 2], each = n_i)
b3 <- rep(MVnorm[, 3], each = n_i) # random effect of time^2
b_4 <- 0.2 # fixed effect of time^2
# update longitudinal
linPred <- b_0 + b1 + (b_1 + b2) * time + b_2 * trtY + b_3 * trtY * time + (b_4 + b3) * f1(time)
Y <- rnorm(n_i * n, linPred, b_e)
datJM_Lt2 <- data.frame(id, time, Y, trt = trtY)
gap2 <- 0.01
mestime2 <- seq(0, fw, gap2)
time2 <- rep(mestime2, n)
n_i2 <- length(mestime2)
trt2 <- rep(trt, each = n_i2)
# b12 <- rep(MVnorm[, 1], each = n_i2)
b22 <- rep(MVnorm[, 2], each = n_i2)
b32 <- rep(MVnorm[, 3], each = n_i2)
# Evaluate the linear predictor on the fine grid for survival simulation => time2 instead of time
CS2_i <- (b_1 + b22) + b_3 * trt2 + 2 * (b_4 + b32) * time2 # slope of the linear predictor

DatTmp <- permlgorithm(n, n_i2, Xmat = matrix(CS2_i, ncol = 1), betas = c(alpha))
DatTmp2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = TRUE)),
  c("Id", "Event", "Stop")]

datJM5_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop + 1],
  eventIndic = DatTmp2$Event, trt = trt)

datJM5_L <- datJM_Lt2[-unlist(sapply(1:n, function(x)
  which(datJM_Lt2$time >= datJM5_S$eventTimes[datJM5_S$id == x] &
    datJM_Lt2$id == x))), ]
```

We fit the model with `assoc = "CS"` and include the new quadratic function in the longitudinal formula:

```
M_jmCS2 <- joint(
  formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong = Y ~ time * trt + f1(time) + (1 + time + f1(time) | id),
```

```

dataSurv = datJM5_S, dataLong = datJM5_L,
id = "id", timeVar = "time", assoc = "CS",
control = list(int.strategy = "eb")
)

summary(M_jmCS2, sdcor = TRUE)

## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    1.0477 0.0350    0.9791    1.0477    1.1163
## time_L1        -0.9685 0.0552   -1.0767   -0.9685   -0.8603
## trt_L1          0.9423 0.0506    0.8432    0.9423    1.0414
## ftime_L1        0.2029 0.0401    0.1244    0.2029    0.2814
## time:trt_L1     0.5180 0.0839    0.3534    0.5180    0.6825
## Res. err. (sd)  0.5062 0.0042    0.4980    0.5062    0.5145
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    0.4996 0.0220    0.4578    0.4993    0.5438
## time_L1         0.5604 0.0427    0.4826    0.5594    0.6465
## ftime_L1        0.6542 0.0381    0.5821    0.6528    0.7315
## Intercept_L1:time_L1 0.4530 0.0851    0.2708    0.4582    0.5994
## Intercept_L1:ftime_L1 0.2195 0.0675    0.0852    0.2196    0.3492
## time_L1:ftime_L1   0.3920 0.0875    0.2069    0.3979    0.5491
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.8549 0.1458    0.5996    0.8448    1.1716
##
## Association longitudinal - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## CS_L1_S1 0.8998 0.0655    0.7719    0.8995    1.0298
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -18892.80                                -18884.24
##
## Deviance Information Criterion: 14496.22
## Widely applicable Bayesian information criterion: 14562.87
## Computation time: 19.74 seconds

```

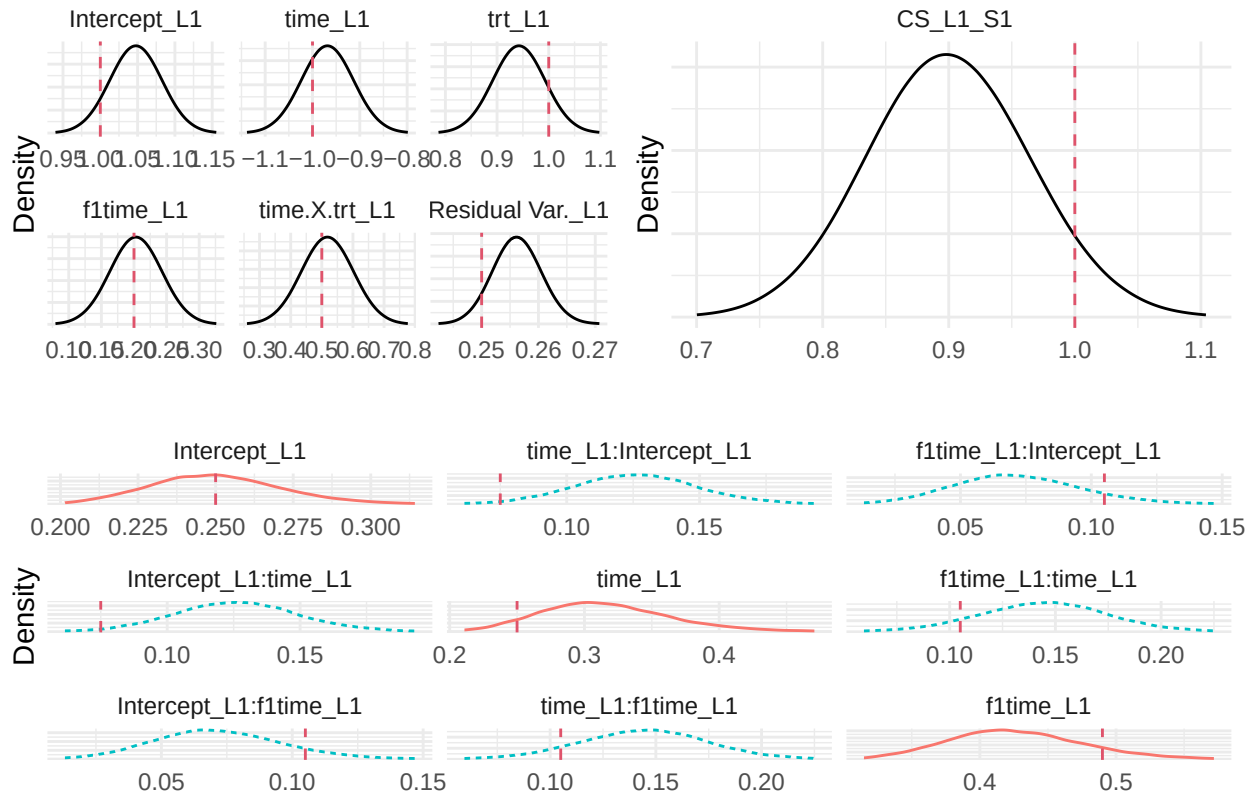
The model accurately estimates the association parameter (CS_L1_S1). A positive value confirms that an increasing trend increases the risk of the event at that specific time.

We can visualize the recovery of the parameters including the new variance component for the quadratic random effect.

```

Plots_jm <- plot(M_jmCS2)
TV1 <- c(b_0, b_1, b_2, b_4, b_3, b_e^2)
TV2 <- c(Sigma)
TV3 <- c(alpha)
ggarrange(ggarrange(eval(parse(text = P1)),
                     eval(parse(text = P3)), ncol = 2),
          eval(parse(text = P2)), nrow = 2)

```



Priors for the association parameters

The association parameters are treated as fixed effects (but internally parametrized as hyperparameters!), assigned a zero-centered Gaussian prior. A few control arguments set the defaults.

- `priorSRE_ind` (default `list(mean = 0, prec = 1)`): prior for the `SRE_ind` association. It has its own argument because this structure links time-constant components, unlike the structures below which can change over time.
- `priorAssoc` (default `list(mean = 0, prec = 0.01)`): shared prior for the `SRE`, `CV` and `CS` associations.
- `assocInit` (default 0.1): starting value for the association parameters. The effect on the final fit is negligible in INLA.

The defaults are weak, so the data drives the estimate. A tighter prior centered at zero pulls the estimate toward no association, which can be a useful sensitivity check.

Dynamic predictions

Personalised dynamic prediction: given a patient's observed longitudinal history up to time t , the model updates the posterior distribution of their latent trajectory and derives both their future biomarker trajectory and their survival probability.

We select two patients with different observed profiles:

```
patient_ids <- datJM3_S$id[1:2]
NewData <- datJM3_L[datJM3_L$id %in% patient_ids & datJM3_L$time <= 1, ];NewData
```

```
##   id time      Y trt
```

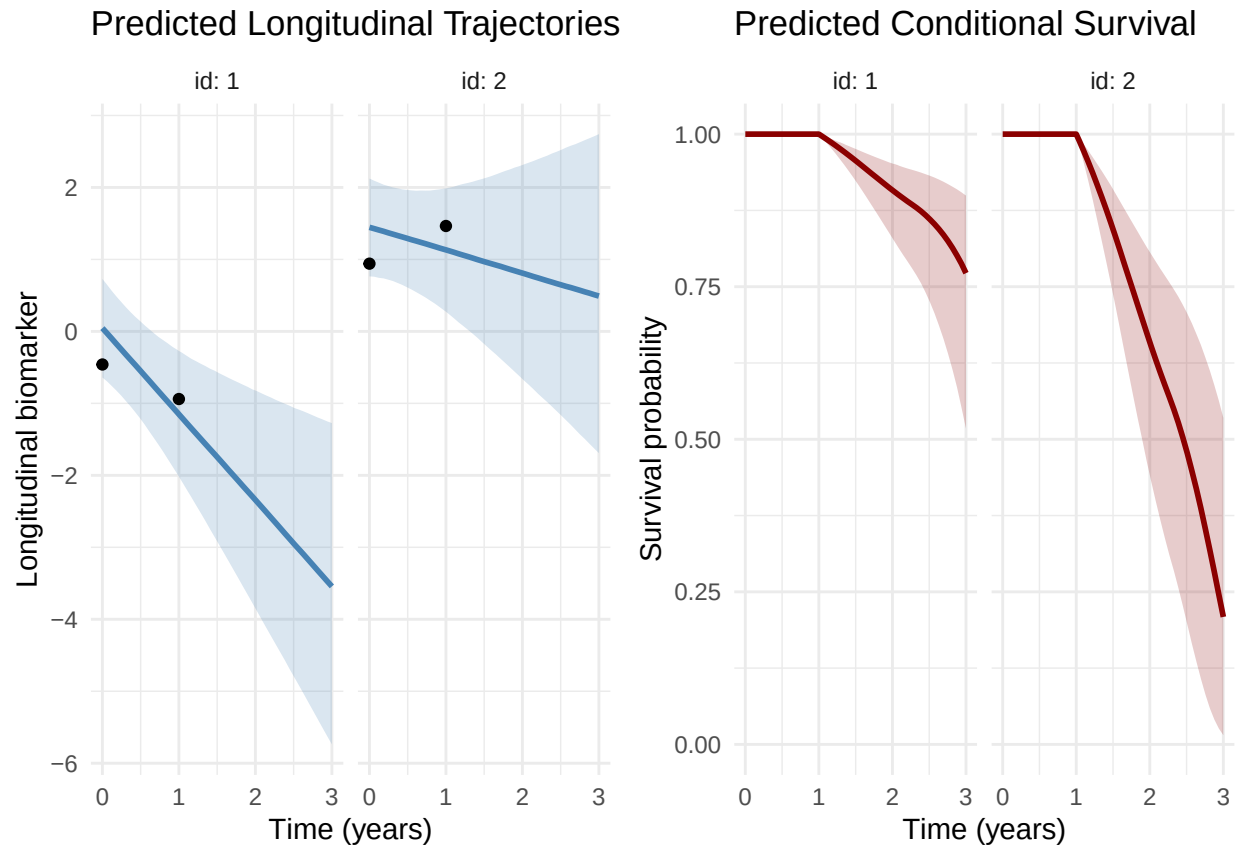
```
## 1 1 0 -0.4599131 0
## 2 1 1 -0.9396970 0
## 7 2 0 0.9412211 1
## 8 2 1 1.4645422 1
```

```
P_dynamic <- predict(
  M_jmCV,
  newData = NewData,
  horizon = 3,
  survival = TRUE,
  id = "id"
)
```

```
p_traj <- ggplot(P_dynamic$PredL, aes(x = time, y = quant0.5)) +
  facet_wrap(~id, labeller = label_both) +
  geom_ribbon(aes(ymin = quant0.025, ymax = quant0.975),
    alpha = 0.2, fill = "steelblue", color = NA) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(data = NewData, aes(x = time, y = Y), size = 1.5) +
  labs(x = "Time (years)", y = "Longitudinal biomarker",
    title = "Predicted Longitudinal Trajectories") +
  theme_minimal()
```

```
p_surv <- ggplot(P_dynamic$PredS, aes(x = time, y = Surv_quant0.5)) +
  facet_wrap(~id, labeller = label_both) +
  geom_ribbon(aes(ymin = Surv_quant0.025, ymax = Surv_quant0.975),
    alpha = 0.2, fill = "darkred", color = NA) +
  geom_line(color = "darkred", linewidth = 1) +
  labs(x = "Time (years)", y = "Survival probability",
    title = "Predicted Conditional Survival") +
  ylim(0, 1) +
  theme_minimal()
```

```
ggarrange(p_traj, p_surv, ncol = 2)
```



The model uses the observed biomarker history to infer each patient's latent trajectory and translates that trajectory into a survival probability through the association.

Updating predictions at different landmark times

A strength of joint models in clinical practice is that predictions can be **updated** as new measurements arrive. We demonstrate this for one patient: first predicting from year 1, then updating the prediction at year 2.

```
patient_data <- datJM3_L[datJM3_L$id == 2, ]

P_year1 <- predict(
  M_jmCV,
  newData = patient_data[patient_data$time<=1, ],
  horizon = 3, survival = TRUE, id = "id"
)

P_year2 <- predict(
  M_jmCV,
  newData = patient_data[patient_data$time<=2, ],
  horizon = 3, survival = TRUE, id = "id"
)

long_df <- rbind(data.frame(P_year1$PredL, Landmark = "Data up to Year 1"),
  data.frame(P_year2$PredL, Landmark = "Data up to Year 2"))

update_df <- rbind(data.frame(P_year1$PredS, Landmark = "Data up to Year 1"),
  data.frame(P_year2$PredS, Landmark = "Data up to Year 2"))
```

```

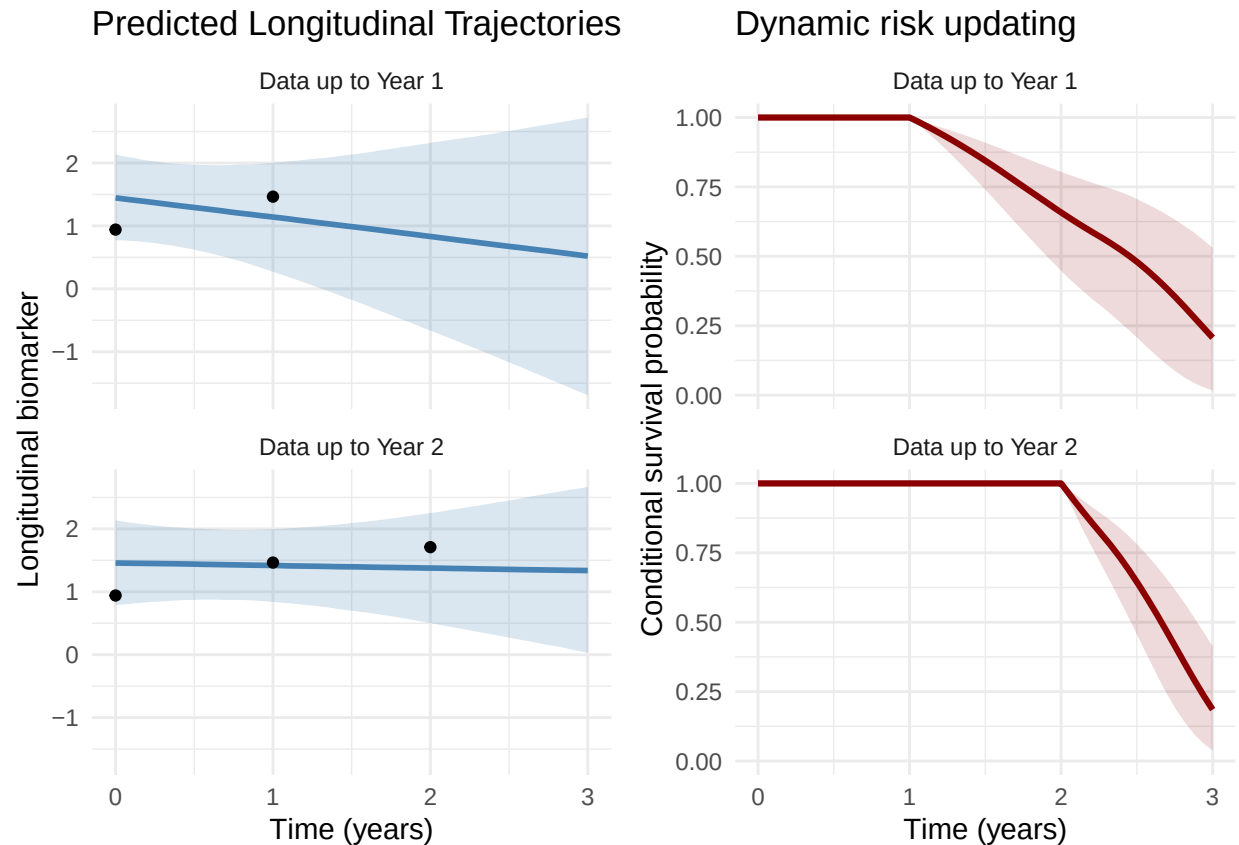
observed_df <- rbind(
  data.frame(patient_data[patient_data$time<=1, ], Landmark = "Data up to Year 1"),
  data.frame(patient_data[patient_data$time<=2,], Landmark = "Data up to Year 2")
)

p_traj <- ggplot(long_df, aes(x = time, y = quant0.5)) +
  facet_wrap(~Landmark, ncol = 1) +
  geom_ribbon(aes(ymin = quant0.025, ymax = quant0.975),
    alpha = 0.2, fill = "steelblue", color = NA) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(data = observed_df, aes(y = Y), size = 1.5) +
  labs(x = "Time (years)", y = "Longitudinal biomarker",
    title = "Predicted Longitudinal Trajectories") +
  theme_minimal()

p_surv <- ggplot(update_df, aes(x = time, y = Surv_quant0.5)) +
  facet_wrap(~Landmark, ncol = 1) +
  geom_ribbon(aes(ymin = Surv_quant0.025, ymax = Surv_quant0.975),
    alpha = 0.15, fill = "darkred", color = NA) +
  geom_line(color = "darkred", linewidth = 1.1) +
  labs(x = "Time (years)", y = "Conditional survival probability",
    title = "Dynamic risk updating") +
  ylim(0, 1) +
  theme_minimal()

ggarrange(p_traj, p_surv, ncol = 2)

```



This updating mechanism, conditioning on the entire observed history, is what makes joint model predictions *dynamic*.

Multivariate joint models

```
set.seed(1)
n = 500 # number of individuals
fw <- 5 # follow-up duration
b_0 = 1 # intercept
b_1 = -1 # slope
b_2 = 1 # trt
b_3 = 0.5 # trt x slope
b_e = 0.5 # residual error
mestime = seq(0, fw, 1) # measurement times
n_i <- length(mestime)
time = rep(mestime, n) # time column
id <- rep(1:n, each = n_i)
b1SD = 1 # random intercept standard deviation
b2SD = 0.7 # random slope standard deviation
cor_bintslo = 0.6 # correlation between the two random effects
cov_bintslo <- b1SD * b2SD * cor_bintslo
Sigma = matrix(c(b1SD^2, cov_bintslo,
                 cov_bintslo, b2SD^2), ncol = 2)
MVnorm <- rmvnorm(n, rep(0, 2), Sigma)
b1 <- rep(MVnorm[, 1], each = n_i) # random intercepts
```

```

b2 <- rep(MVnorm[, 2], each = n_i) # random slopes
trt <- rbinom(n, 1, 0.5)
trtY <- rep(trt, each = length(mestime))
linPred <- b_0 + b1 + (b_1 + b2) * time + b_2 * trtY + b_3 * trtY * time
Y <- rnorm(n_i * n, linPred, b_e)
datJM_Lt <- data.frame(id, time, Y, trt = trtY)
baseScale <- 0.1
alpha_1 <- 0.5 # random intercept effect on survival
alpha_2 <- 0.3 # random slope effect on survival

```

The same random effects feed a fine time grid for the permutation algorithm.

```

gap2 <- 0.01 # fine grid for PermAlgo
mestime2 <- seq(0, fw, gap2) # measurement times
time2 <- rep(mestime2, n)
n_i2 <- length(mestime2)
trt2 <- rep(trt, each = n_i2)
b12 <- rep(MVnorm[, 1], each = n_i2)
b22 <- rep(MVnorm[, 2], each = n_i2)

```

Two markers, one event

Say tumor size is not the only signal we collect, we also count new lesions at each visit.

Tumor size is continuous. Lesion counts are counts.

Different scales, different families.

We can fit two separate joint models, one per marker.

- That ignores the link between the two markers.
- A patient whose tumor grows fast may also gain lesions fast.
- Separate fits cannot see that shared patient-level signal.
- It also makes us pick which marker drives the risk, instead of letting both speak.

The solution

One model. Two longitudinal submodels under one likelihood.

- `formLong` becomes a list of formulas.
- `family` becomes a vector, one entry per marker.
- `assoc` becomes a vector, one association per marker.

The structure we are simulating from:

$$\begin{aligned}
 E[Y_{1i}(t)] &= \eta_{1i}(t) \quad (\text{continuous, Gaussian}) \\
 \log E[Y_{2i}(t)] &= \eta_{2i}(t) \quad (\text{counts, Poisson}) \\
 \lambda_i(t) &= \lambda_0(t) \exp\{\alpha_1 \eta_{1i}(t) + \alpha_2 \eta_{2i}(t)\}
 \end{aligned}$$

- Two current-value links, one per marker.
- Each exponentiated alpha is its own hazard ratio, the risk carried by that marker once we hold the other one fixed.

Simulate the two markers

```

set.seed(3)
CV_i1 <- b_0 + b12 + b_1*time2 + b_2*trt2 + b_3*trt2*time2
CV_i1L <- b_0 + b1 + b_1*time + b_2*trtY + b_3*trtY*time
CV_i2 <- b_0 + b22 + b_1*time2 + b_2*trt2 + b_3*trt2*time2
CV_i2L <- b_0 + b2 + b_1*time + b_2*trtY + b_3*trtY*time
DatTmp <- permalgorithm(n, n_i2, Xmat=matrix(c(CV_i1, CV_i2), ncol=2),
                        betas=c(alpha_1, alpha_2)) # association
DatTmp2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = T)),
                  c("Id", "Event", "Stop")]
datJM7_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop+1],
                      eventIndic = DatTmp2$Event, trt = trt)
datJM_Lt$Y1 <- rnorm(n_i*n, CV_i1L, b_e) # first outcome Gaussian
datJM_Lt$Y2 <- rpois(n_i*n, exp(CV_i2L)) # second outcome Poisson
datJM7_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which(datJM_Lt$time>=datJM7_S$eventTimes[datJM7_S$id==x] &
    datJM_Lt$id==x))),]

```

What this block does:

- Two markers from correlated random intercepts.
- The first is Gaussian, the second is counts.
- Both affect the risk of death through their current value.

Fit it

```

M_jmCV2 <- joint(formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
                 formLong = list(Y1 ~ time * trt + (1 | id),
                                Y2 ~ time * trt + (1 | id)),
                 dataSurv = datJM7_S, dataLong = datJM7_L,
                 family = c("gaussian", "poisson"), id = "id",
                 timeVar = "time", assoc = c("CV", "CV"))
summary(M_jmCV2)

```

```

## Longitudinal outcome (L1, gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    1.0171 0.0710    0.8778    1.0171    1.1564
## time_L1        -0.9906 0.0213   -1.0325   -0.9906   -0.9487
## trt_L1          0.9000 0.1029    0.6982    0.9000    1.1018
## time:trt_L1     0.5409 0.0393    0.4638    0.5409    0.6178
## Res. err. (variance) 0.2321 0.0135    0.2069    0.2317    0.2599
##
## Random effects variance-covariance (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1  1.1246 0.0795    0.9769    1.1218    1.2893
##
## Longitudinal outcome (L2, poisson)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L2  1.0203 0.0581    0.9053    1.0206    1.1334
## time_L2      -0.9702 0.0517   -1.0718   -0.9701   -0.8692
## trt_L2        0.8909 0.0777    0.7388    0.8908    1.0439
## time:trt_L2   0.4616 0.0622    0.3398    0.4614    0.5839
##
## Random effects variance-covariance (L2)
##               mean      sd 0.025quant 0.5quant 0.975quant

```

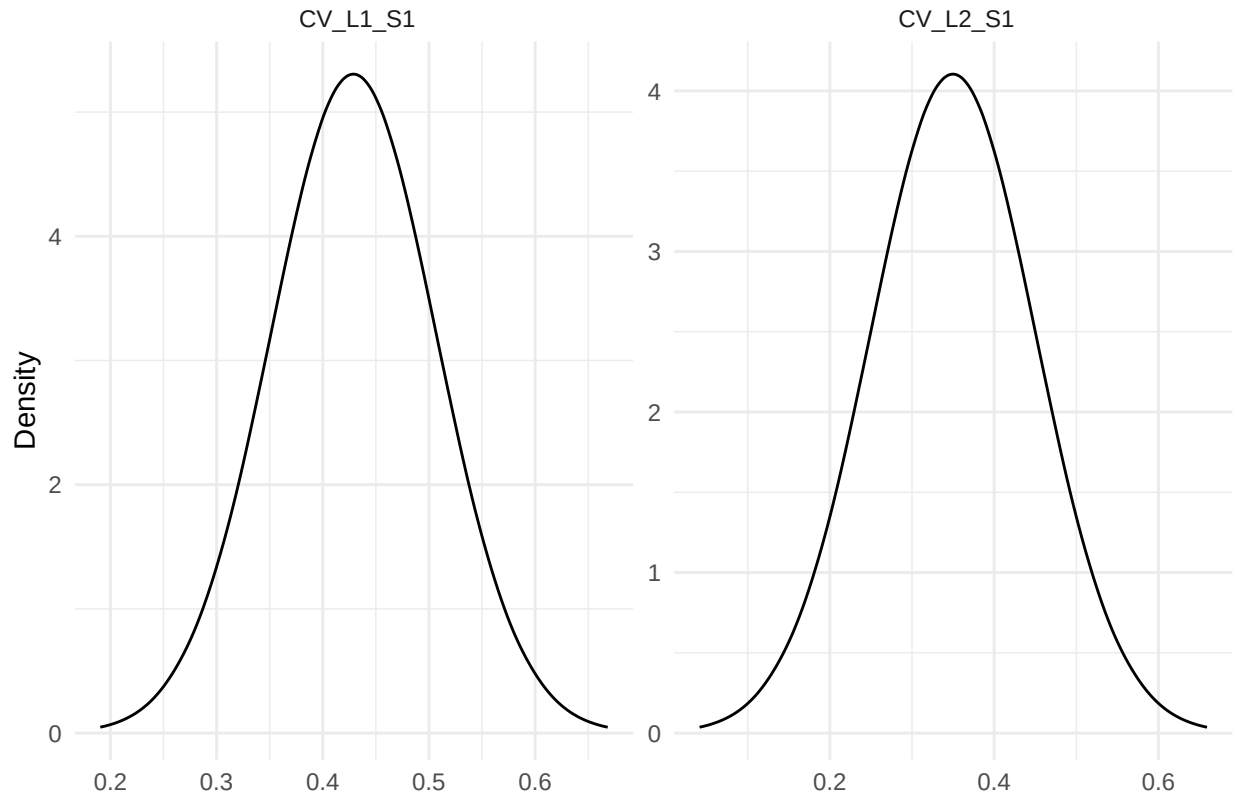
```
## Intercept_L2 0.524 0.0452      0.4411    0.522    0.6187
##
## Survival outcome
##              mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.6339 0.2381    0.2917    0.5923    1.219
##
## Association longitudinal - survival
##              mean      sd 0.025quant 0.5quant 0.975quant
## CV_L1_S1 0.4291 0.0778    0.2760    0.429    0.5824
## CV_L2_S1 0.3500 0.1006    0.1521    0.350    0.5481
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##              -25557.07                      -25553.04
##
## Deviance Information Criterion: 7883.308
## Widely applicable Bayesian information criterion: 7841.384
## Computation time: 8.66 seconds
```

Reading the summary, top to bottom:

- Marker one first, then marker two.
- Then the survival part.
- Then two association parameters, one per marker.
- Both sit near the values I simulated from, 0.5 and 0.3.

The two association posteriors, side by side:

```
jm_plots <- plot(M_jmCV2)
jm_plots$Associations
```



Let the markers inform each other

We built the two markers from correlated random intercepts. The fit above kept them independent and ignored that link.

- One switch turns it on, `corLong = TRUE`.
- Now the random intercepts share a covariance.

```
M_jmCV3 <- joint(formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong = list(Y1 ~ time * trt + (1 | id),
    Y2 ~ time * trt + (1 | id)),
  dataSurv = datJM7_S, dataLong = datJM7_L, corLong = TRUE,
  family = c("gaussian", "poisson"), id = "id",
  timeVar = "time", assoc = c("CV", "CV"))
summary(M_jmCV3, sdcor = TRUE)
```

```
## Longitudinal outcome (L1, gaussian)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1  1.0177 0.0712   0.8781   1.0177   1.1573
## time_L1      -0.9868 0.0213  -1.0286  -0.9868  -0.9451
## trt_L1        0.8998 0.1031   0.6976   0.8998   1.1021
## time:trt_L1   0.5499 0.0391   0.4732   0.5499   0.6265
## Res. err. (sd) 0.4821 0.0140   0.4555   0.4818   0.5105
##
## Longitudinal outcome (L2, poisson)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L2  1.0233 0.0581   0.9085   1.0236   1.1364
```

```
## time_L2      -0.9541 0.0515    -1.0554 -0.9539    -0.8535
## trt_L2       0.8863 0.0777     0.7342  0.8862     1.0392
## time:trt_L2  0.4605 0.0618     0.3397  0.4604     0.5821
##
## Random effects standard deviation / correlation
##              mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1      1.0640 0.0377      0.9910  1.0629    1.1387
## Intercept_L2       0.7261 0.0306      0.6682  0.7253    0.7875
## Intercept_L1:Intercept_L2 0.5969 0.0395      0.5156  0.5987    0.6686
##
## Survival outcome
##              mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.7831 0.1452      0.5372  0.7699    1.1068
##
## Association longitudinal - survival
##              mean      sd 0.025quant 0.5quant 0.975quant
## CV_L1_S1 0.4022 0.0907      0.2255  0.4015    0.5826
## CV_L2_S1 0.3802 0.1182      0.1451  0.3811    0.6103
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##              -25476.26                      -25471.29
##
## Deviance Information Criterion: 7823.73
## Widely applicable Bayesian information criterion: 7794.905
## Computation time: 13.69 seconds
```

The takeaway:

- The estimated correlation recovers the true 0.6.
- A patient high on one marker tends to be high on the other.

Two markers, two events

Death is not the only thing that can happen.

- A patient can be hospitalized.
- Disease can recur.
- Different events may be informed by different markers.

Example:

- Maybe the current tumor level drives death.
- Maybe a patient's baseline frailty, the random intercept, drives the second event.

Separate survival models per event cannot share the patient-level signal across events. The random effects are what carry a patient's frailty from one event into the other.

The solution

- `formSurv` becomes a list, one formula per event.
- `assoc` becomes a list, one element per marker.
- Inside each element, a vector with one entry per event.

The structure we are simulating from:

$$\lambda_{1i}(t) = \lambda_{10}(t) \exp\{\alpha_{11} \eta_{1i}(t) + \alpha_{12} \eta_{2i}(t)\}$$

$$\lambda_{2i}(t) = \lambda_{20}(t) \exp\{\alpha_{21} b_{1i0}\}$$

- Event one depends on both markers at their current value.
- Event two depends on only the baseline level of marker one, its random intercept.
- “CV” for the current-value link, “SRE_ind” for sharing a single random effect.

(here we assume no competing risk and no multi-state for simplicity).

Simulate the second event

This block builds on the previous simulation. It keeps the event-one times and adds a second event driven by the random intercept of marker one.

```
set.seed(3)
alpha_21 <- 0.2
DatTmp <- permalgorithm(n, n_i2, Xmat=matrix(b12, ncol=1), betas=alpha_21)
DatTmp2$Event2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = T)),
                           "Event"]
DatTmp2$Stop2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = T)),
                        "Stop"]
datJM8_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop+1],
                      eventTimes2 = mestime2[DatTmp2$Stop2+1],
                      eventIndic = DatTmp2$Event,
                      eventIndic2 = DatTmp2$Event2, trt = trt)
datJM8_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which((datJM_Lt$time>=datJM8_S$eventTimes[datJM8_S$id==x] |
        datJM_Lt$time>=datJM8_S$eventTimes2[datJM8_S$id==x]) &
        datJM_Lt$id==x))),]
```

Fit it

```
M_jmCV4 <- joint(formSurv = list(inla.surv(eventTimes, eventIndic) ~ -1,
                                inla.surv(eventTimes2, eventIndic2) ~ -1),
                 formLong = list(Y1 ~ time * trt + (1 | id),
                                Y2 ~ time * trt + (1 | id)),
                 dataSurv = datJM8_S, dataLong = datJM8_L, corLong = TRUE,
                 family = c("gaussian", "poisson"), id = "id",
                 timeVar = "time", assoc = list(c("CV", "SRE_ind"),
                                                  c("CV", "")))
summary(M_jmCV4, sdcor = TRUE)
```

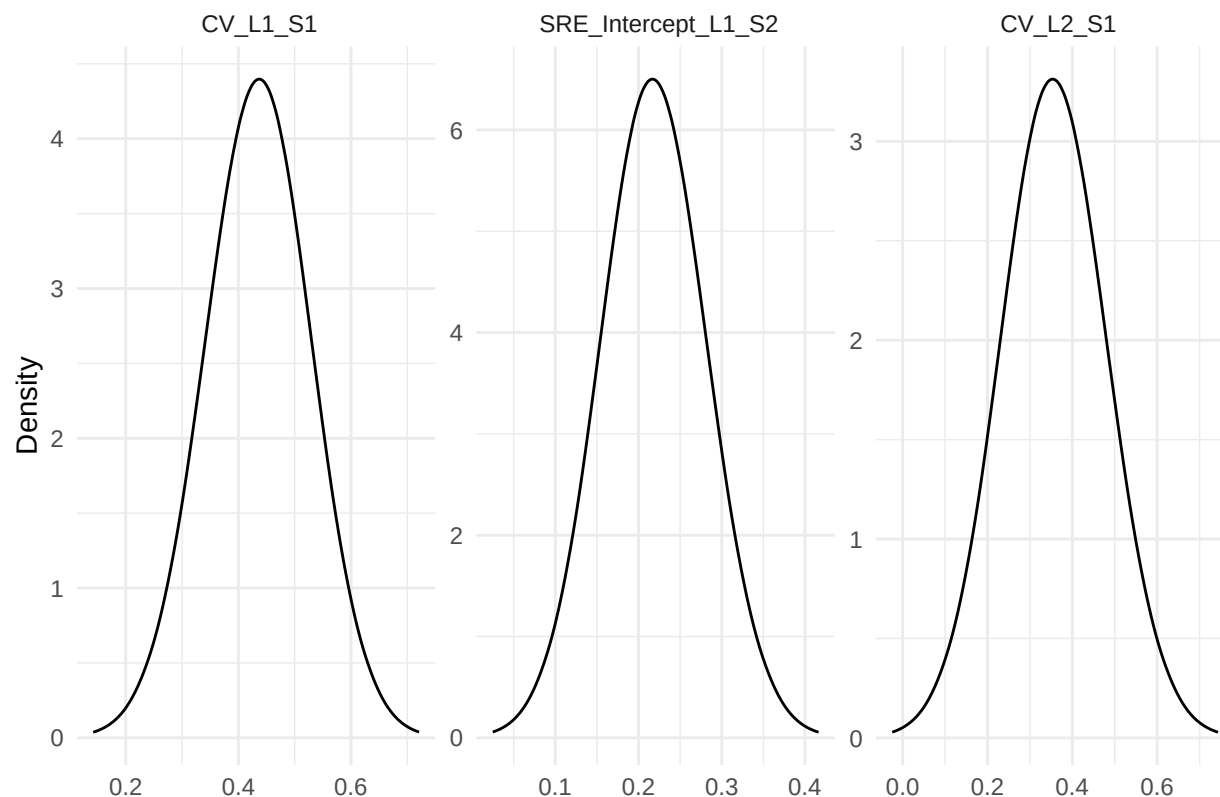
```
## Longitudinal outcome (L1, gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1   1.0289 0.0706   0.8903   1.0289   1.1675
## time_L1        -0.9995 0.0344  -1.0669  -0.9996  -0.9321
## trt_L1          0.8841 0.1016   0.6848   0.8841   1.0833
## time:trt_L1     0.5763 0.0616   0.4553   0.5763   0.6972
## Res. err. (sd) 0.4875 0.0193   0.4508   0.4871   0.5268
##
## Longitudinal outcome (L2, poisson)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L2   1.0377 0.0579   0.9231   1.0381   1.1505
## time_L2        -0.9644 0.0691  -1.1010  -0.9641  -0.8300
## trt_L2          0.8731 0.0772   0.7220   0.8729   1.0248
## time:trt_L2     0.4398 0.0836   0.2773   0.4393   0.6052
##
## Random effects standard deviation / correlation
```

```
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1      1.0477 0.0392      0.9763  1.0459    1.1297
## Intercept_L2      0.7200 0.0323      0.6594  0.7187    0.7861
## Intercept_L1:Intercept_L2 0.5845 0.0411      0.5041  0.5853    0.6584
##
## Survival outcome (S1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.7888 0.1461      0.5327  0.7792    1.1051
##
## Survival outcome (S2)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S2 0.3738 0.1144      0.2033  0.3554    0.6499
##
## Association longitudinal - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## CV_L1_S1          0.4346 0.0939      0.2484  0.4351    0.6180
## SRE_Intercept_L1_S2 0.2185 0.0635      0.0944  0.2181    0.3445
## CV_L2_S1          0.3564 0.1246      0.1127  0.3559    0.6034
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -25976.71                      -25969.94
##
## Deviance Information Criterion: 8940.258
## Widely applicable Bayesian information criterion: 8889.567
## Computation time: 39.94 seconds
```

- Marker one: CV with event one, SRE_ind with event two.
- Marker two: CV with event one, nothing with event two.
- Empty string = no link.

The three association posteriors:

```
jm_plots2 <- plot(M_jmCV4)
jm_plots2$Associations
```



- Two for event one, the current values of both markers.
- One for event two, the shared random intercept.

Two more association structures

The main session covered SRE_ind, SRE, CV and CS. Each scales a shared component by a constant.

Think about these situations:

- Relationship between number of cigarettes smoked and the risk of lung cancer
- Relationship between BMI and all cause mortality

Two new association structures can deal with these tasks:

- Cumulative value. The whole (or part of) history of the marker, not just today.
- Non-linear. The effect of the marker can bend, not just scale.

Consider the current value of a single marker (evaluated at a fine grid):

```
CV_i <- b_0 + b12 + (b_1 + b22) * time2 + b_2 * trt2 + b_3 * trt2 * time2
```

Cumulative value, the area under the curve

Cumulative exposure. A patient who sat at a high tumor size for a year has a different risk profile compared to one who spiked only yesterday.

The structure:

$$\lambda_i(t) = \lambda_0(t) \exp \left(\alpha \cdot \int_0^t \eta_i(s) ds \right)$$

- The hazard scales the integral of the marker, not its current level.
- One parameter, alpha, the cumulative exposure effect.

This one has no keyword. We build it on top of a current-value model with a weight matrix.

First the cumulative values and the event times that depend on them.

```
set.seed(1)
alpha <- 0.1
CUM_CV_i <- rep(NA, length(CV_i))
for (i in 1:n) {
  CUM_CV_i[(n_i2*(i-1)+1):(n_i2*i)] <- cumsum(
    c(0, CV_i[(n_i2*(i-1)+1):(n_i2*i)-1]) * gap2)
}
DatTmp <- permlgorithm(n, n_i2, Xmat = matrix(CUM_CV_i, ncol = 1), betas = alpha)
DatTmp2 <- DatTmp[which(!duplicated(DatTmp$Id, fromLast = T)),
  c("Id", "Event", "Stop")]
dataAUC_S <- data.frame(id = 1:n, eventTimes = mestime2[DatTmp2$Stop+1],
  eventIndic = DatTmp2$Event, trt = trt)
dataAUC_L <- datJM_Lt[-unlist(sapply(1:n, function(x)
  which(datJM_Lt$time >= dataAUC_S$eventTimes[dataAUC_S$id == x] &
    datJM_Lt$id == x))),]
```

Set up a current-value model but hold the fit with run = FALSE.

```
M_auc0 <- joint(formSurv = inla.surv(eventTimes, eventIndic) ~ -1,
  formLong = Y ~ time * trt + (1 + time | id),
  dataSurv = dataAUC_S, dataLong = dataAUC_L,
  id = "id", timeVar = "time", assoc = "CV",
  run = FALSE)
```

The weights block turns each interval's value into a running integral, scaled by the length of every interval up to t .

```
w <- na.omit(M_auc0$.args$data$E..coxph)
w.idx <- na.omit(M_auc0$.args$data$baseline1.hazard.idx)
w.size <- length(M_auc0$.args$data$baseline1.hazard.values)
maxSize <- w.size == length(M_auc0$.args$data$baseline1.hazard.values)
nA <- length(w)
nT <- length(M_auc0$.args$data$Intercept_L1)
j.acc <- 0
ijx <- list(i = rep(0L, w.size * nA),
  j = rep(0L, w.size * nA),
  x = rep(0.0, w.size * nA))
nacc <- 0
for (i in 1:nA) {
  iw <- tail(1:w.idx[i], w.size)
  ni <- length(iw)
  if ((i > 1) & (w.idx[i] == 1)) j.acc <- j.acc + w.idx[i-1]
  jj <- j.acc + iw
  w0 <- w[jj]
  fixw <- (!maxSize & tail(w0, 1) < (0.99 * w0[1]))
  if ((length(jj) > 1) && fixw) {
    iw <- tail(1:w.idx[i], w.size + 1)
```

```

    ni <- length(iw)
    jj <- j.acc + iw
    w0 <- w[jj]
    w0[1] <- w0[1] - tail(w0, 1)
  }
  ijk[i[nacc + 1:ni]] <- i
  ijk[j[nacc + 1:ni]] <- jj
  ijk[x[nacc + 1:ni]] <- w0
  nacc <- nacc + ni
}
A0 <- sparseMatrix(i = ijk$i[1:nacc], j = ijk$j[1:nacc], x = ijk$x[1:nacc])
Adata <- rbind(sparseMatrix(i = 1, j = 1, x = 0, dims = c((nT - nA), nA)), A0)
inla.as.sparse(A0[1:10, 1:10])

```

```
## 10 x 10 sparse Matrix of class "dgTMatrix"
```

```
##
## [1,] 0.316 . . . . . . . . .
## [2,] 0.316 0.316 . . . . . . .
## [3,] 0.316 0.316 0.316 . . . . .
## [4,] 0.316 0.316 0.316 0.316 . . . .
## [5,] 0.316 0.316 0.316 0.316 0.316 . . .
## [6,] 0.316 0.316 0.316 0.316 0.316 0.316 . .
## [7,] 0.316 0.316 0.316 0.316 0.316 0.316 0.316 .
## [8,] 0.316 0.316 0.316 0.316 0.316 0.316 0.316 0.268 .
## [9,] . . . . . . . . 0.316 .
## [10,] . . . . . . . . 0.316 0.316
```

Each row is one follow-up interval. It carries the running sum of the marker over all earlier intervals, weighted by their length. We drop this matrix into the model through `A.local` and run.

```

M_auc0$.args$data$WCUM <- rep(0, nT)
M_auc0$.args$formula <- as.formula(paste(c("Yjoint ~ -1 ",
  attr(terms(M_auc0$.args$formula), which = "term.labels")[1:9],
  "+ f(CV_L1_S1, WCUM, copy = 'uv1', hyper = list(beta = list(
    fixed = FALSE, param = c(0, 0.01), initial = 0.1)),
    A.local = Adata)"), collapse = "+"))
M_auc <- joint.run(M_auc0)
summary(M_auc)

```

```
## Longitudinal outcome (gaussian)
##
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1      1.0421 0.0694      0.9060      1.0421      1.1782
## time_L1          -0.9631 0.0627     -1.0859     -0.9631     -0.8398
## trt_L1            0.8815 0.1002      0.6850      0.8815      1.0781
## time:trt_L1       0.4678 0.0919      0.2876      0.4677      0.6483
## Res. err. (variance) 0.2695 0.0257      0.2233      0.2681      0.3241
##
## Random effects variance-covariance (L1)
##
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1      1.0131 0.0912      0.8530      1.0089      1.2001
## time_L1          0.4798 0.0565      0.3786      0.4770      0.6035
## Intercept_L1:time_L1 0.4357 0.0740      0.2736      0.4427      0.5564
##
## Survival outcome
##
##           mean      sd 0.025quant 0.5quant 0.975quant
```

```
## Baseline risk (variance)_S1 0.1253 0.0754      0.0308   0.1087      0.3196
##
## Association longitudinal - survival
##          mean      sd 0.025quant 0.5quant 0.975quant
## CV_L1_S1 0.0893 0.0178      0.0549   0.0891      0.1249
##
## log marginal-likelihood (integration)      log marginal-likelihood (Gaussian)
##                                     -13795.50                                     -13790.94
##
## Deviance Information Criterion:  4333.161
## Widely applicable Bayesian information criterion:  4289.576
## Computation time: 17.18 seconds
```

- CV_L1_S1 is now the cumulative exposure effect.

Non-linear association

Every structure so far scales a shared component by one constant.

$$\log HR = \alpha \cdot h(\eta_i(t))$$

- Move the marker up by one unit, the log-risk shifts by the same α everywhere.
- Interpretable, and often enough.
- Tumor size rising is bad, and more is worse. Monotone, so a flat scaling fits.

Body mass index and mortality have a U-shaped association. Both ends raise risk. A flat scaling averages them into no association.

A marker can protect when low and harm when high, with the change per unit growing as the level moves out. A straight line on the log scale might lack flexibility.

Three nested levels, each one more flexible than the last.

Level	assoc	Scaling $f(x)$
Linear	"CV"	α
Quadratic	"L_CV"	$\alpha_1 + \alpha_2 x$
Spline	"NL_CV"	$\alpha_1 + \alpha_2 x + \text{RW2}(x)$

- Linear. One hazard ratio, standard joint model.
- Quadratic. The scaling gains a slope, so the log hazard ratio becomes a parabola. This alone captures a U or an inverted U.
- Spline. A second-order random walk adds a smooth deviation on top of the line. This reaches shapes a linear + parabola cannot.

The prefix identifies the structure:

- Add L_ for the linear scaling, so "CV" becomes "L_CV".
- Add NL_ for the spline, so "CV" becomes "NL_CV".
- The same prefixes apply to the other structures, "L_SRE", "NL_SRE", "L_CS" and so on.

We can compare the models with standard criterion (WAIC, DIC).

Example

One marker with a curved effect, protective when low and harmful when high, with the per-unit effect growing as the value moves out from the center.

```

set.seed(1)

# Config
n <- 500
beta <- c(-1.5, 0.3, 0.5, 0.2)
Sigma <- matrix(c(0.64, 0.072, 0.072, 0.09), 2, 2)
sigma_e <- 0.3
F_assoc <- function(x) x * log(x^2 + 1)^1.5 / 8 # curved effect on the log hazard

# Data generation
time_fine <- seq(0, 10, 0.001)
time_long <- seq(0, 10, 0.2)
trt <- rbinom(n, 1, 0.5)
re <- rmvnorm(n, sigma = Sigma)

# Survival
id_fine <- rep(1:n, each = length(time_fine))
time_fine_rep <- rep(time_fine, n)
trt_fine <- rep(trt, each = length(time_fine))
eta_fine <- beta[1] + re[id_fine, 1] +
  (beta[2] + re[id_fine, 2]) * time_fine_rep +
  beta[3] * trt_fine + beta[4] * time_fine_rep * trt_fine

surv <- permalgorithm(n, length(time_fine),
  Xmat = as.matrix(F_assoc(eta_fine)),
  eventRandom = round(rexp(n, 0.0001) + 1),
  XmatNames = "assoc", betas = 1)
d_surv <- surv[c(which(diff(surv$Id) == 1), nrow(surv)),
  c("Id", "Event", "Stop")]
d_surv$Stop <- pmin(d_surv$Stop, length(time_fine) - 1)
d_surv$time <- time_fine[d_surv$Stop + 1]
d_surv$time[is.na(d_surv$time)] <- 10
d_surv <- data.frame(id = d_surv$Id, time = d_surv$time, event = d_surv$Event)

# Longitudinal
id_long <- rep(1:n, each = length(time_long))
time_long_rep <- rep(time_long, n)
trt_long <- rep(trt, each = length(time_long))
eta_long <- beta[1] + re[id_long, 1] +
  (beta[2] + re[id_long, 2]) * time_long_rep +
  beta[3] * trt_long + beta[4] * time_long_rep * trt_long
y <- rnorm(length(eta_long), eta_long, sigma_e)
d_long <- data.frame(id = id_long, time = time_long_rep, y = y, trt = trt_long)
d_long <- d_long[d_long$time <= d_surv$time[match(d_long$id, d_surv$id)], ]

```

Fit the three levels on the same data.

```

formLong <- list(y ~ time * trt + (1 + time | id))
formSurv <- inla.surv(time, event) ~ -1

# Level 1 - linear
M1 <- joint(formLong = formLong, formSurv = formSurv,
  dataLong = d_long, dataSurv = d_surv,
  id = "id", timeVar = "time",

```

```

    assoc = "CV", basRisk = "rw2",
    control = list(int.strategy = "eb"))

# Level 2 - quadratic
M2 <- joint(formLong = formLong, formSurv = formSurv,
            dataLong = d_long, dataSurv = d_surv,
            id = "id", timeVar = "time",
            assoc = "L_CV", basRisk = "rw2",
            control = list(int.strategy = "eb"))

# Level 3 - spline
M3 <- joint(formLong = formLong, formSurv = formSurv,
            dataLong = d_long, dataSurv = d_surv,
            id = "id", timeVar = "time",
            assoc = "NL_CV", basRisk = "rw2",
            control = list(int.strategy = "eb"))

```

Which level do the data support?

`joint.compare` puts the three side by side on WAIC, with a test on each difference.

```
joint.compare(M1, M2, M3)
```

```

##   model1 model2 dWAIC  se    z    p better sig
## 1     M1     M2  16.1 7.0 2.30 0.022    M2  *
## 2     M1     M3  30.2 9.3 3.24 0.001    M3  **
## 3     M2     M3  14.1 4.8 2.91 0.004    M3  **

```

Reading the table:

- Linear to quadratic is favored. The effect is not flat.
- Quadratic to spline is favored again. A parabola is not enough, the shape needs the random walk.
- The spline wins.

The fitted spline association:

```
summary(M3, sdcor = TRUE)
```

```

## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1  -1.5156 0.0531   -1.6197  -1.5156   -1.4115
## time_L1        0.2904 0.0209    0.2494   0.2904    0.3314
## trt_L1         0.4734 0.0783    0.3199   0.4734    0.6270
## time:trt_L1    0.1845 0.0311    0.1234   0.1845    0.2455
## Res. err. (sd) 0.2999 0.0023    0.2954   0.2999    0.3045
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1    0.8618 0.0288    0.8078   0.8611    0.9209
## time_L1         0.2974 0.0115    0.2762   0.2970    0.3204
## Intercept_L1:time_L1 0.2858 0.0390    0.2119   0.2831    0.3649
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.2003 0.0163    0.172   0.1989    0.2359
##
## Association longitudinal - survival

```

```
##               mean      sd 0.025quant 0.5quant 0.975quant
## Beta0 for NL_CV_L1_S1 0.4004 0.0550      0.2952   0.3994     0.5117
## Beta1 for NL_CV_L1_S1 0.1565 0.1649     -0.1974   0.1664     0.4471
## Beta2 for NL_CV_L1_S1 0.2688 0.0674      0.1431   0.2666     0.4079
## Beta3 for NL_CV_L1_S1 0.0009 0.0591     -0.1101  -0.0009     0.1224
## Beta4 for NL_CV_L1_S1 0.0981 0.0881     -0.0429   0.0889     0.2966
##
## log marginal-likelihood (integration)      log marginal-likelihood (Gaussian)
##               -40294.04                      -40284.56
##
## Deviance Information Criterion: 6333.755
## Widely applicable Bayesian information criterion: 6347.988
## Computation time: 29.68 seconds
```

- Five association parameters. An intercept, a slope, then three for the random walk.

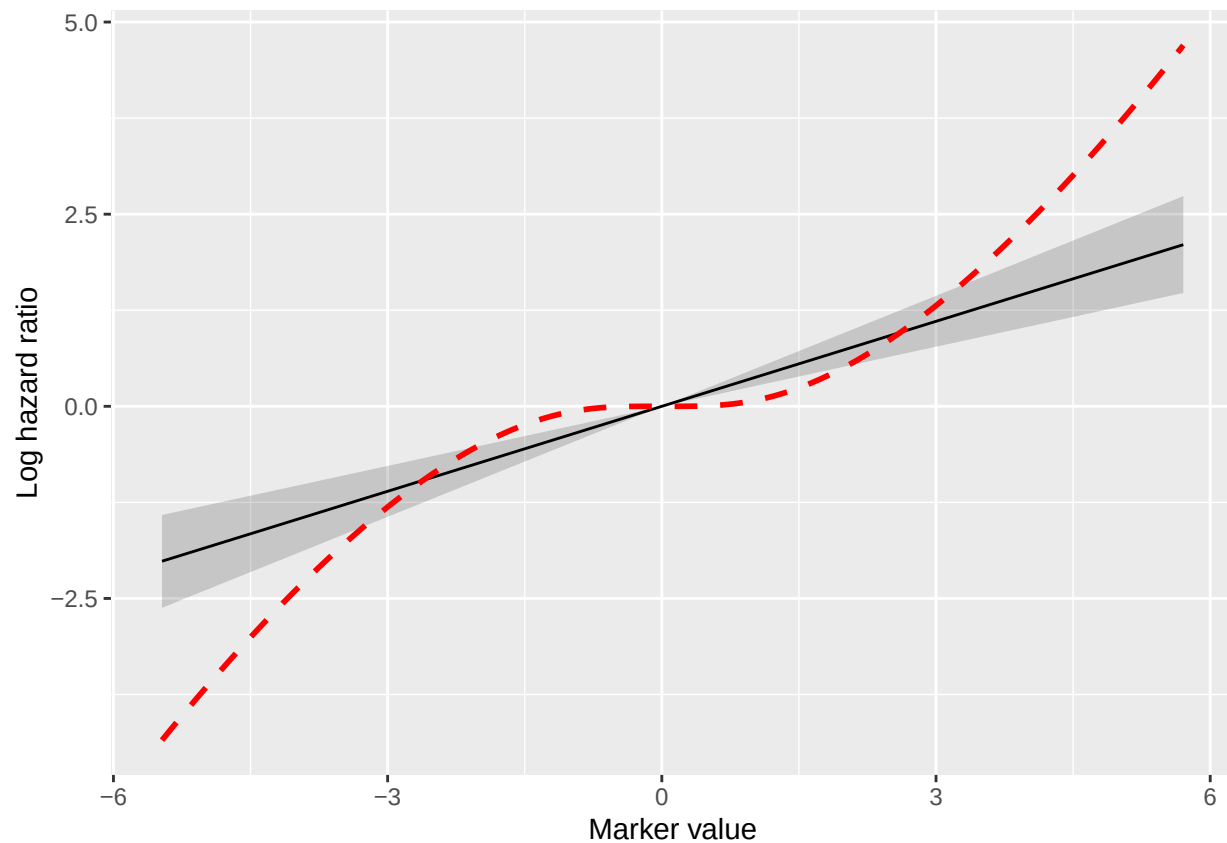
Question. The linear model reports a single hazard ratio. Why can one number mislead for this marker?

Answer. The effect changes sign and steepness across the range. A single value reports an average that fits neither the low end nor the high end.

Recovered effect

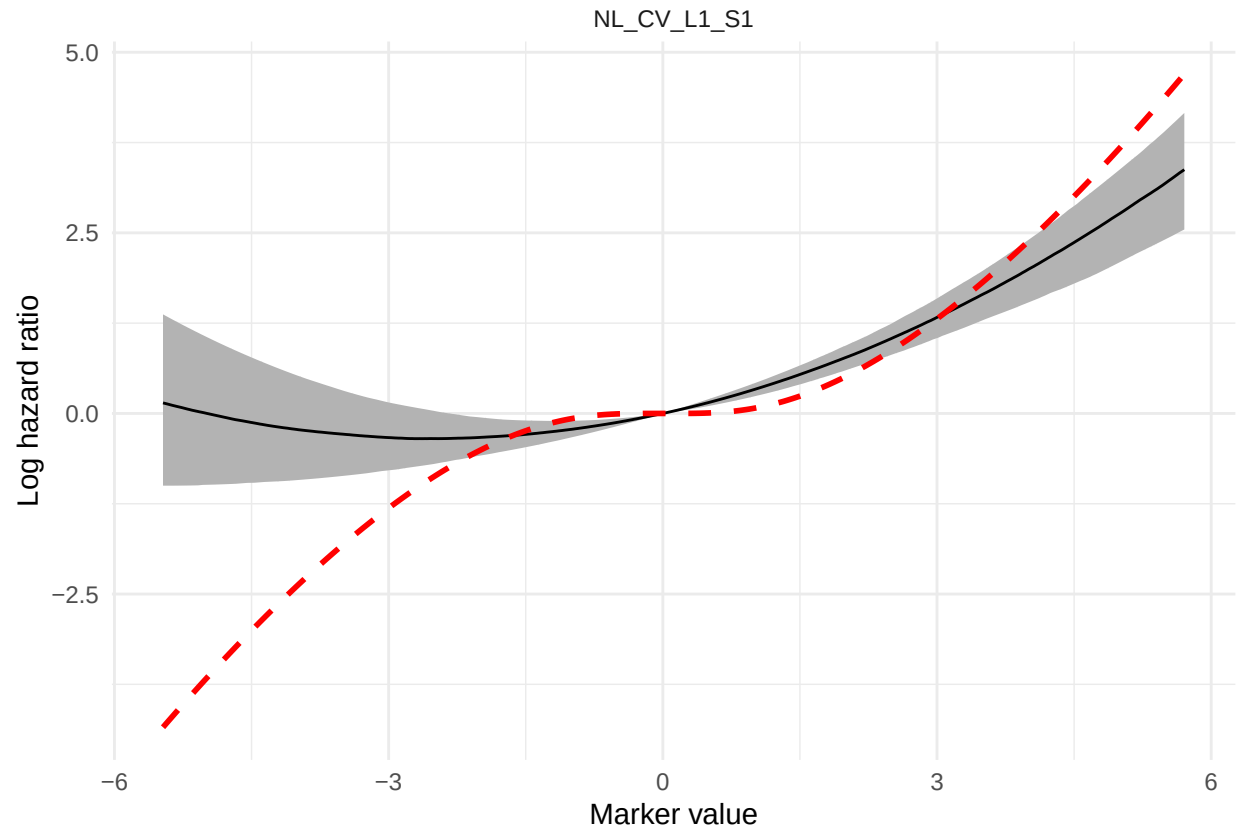
With linear:

```
q <- as.numeric(M1$summary.hyperpar["Beta for CV_L1_S1", 3:5])
grid <- plot(M2)$NL_Associations$data$x
p_lhr <- ggplot(data.frame(x = grid), aes(x)) + geom_ribbon(aes(ymin = pmin(q[1] * x, q[3] * x),
                                ymax = pmax(q[1] * x, q[3] * x)), alpha = 0.2) +
  geom_line(aes(y = q[2] * x))
p_lhr +
  geom_line(data = data.frame(x = grid, y = F_assoc(grid)),
            aes(x = x, y = y),
            colour = "red", linetype = "dashed", linewidth = 1) +
  labs(x = "Marker value", y = "Log hazard ratio")
```



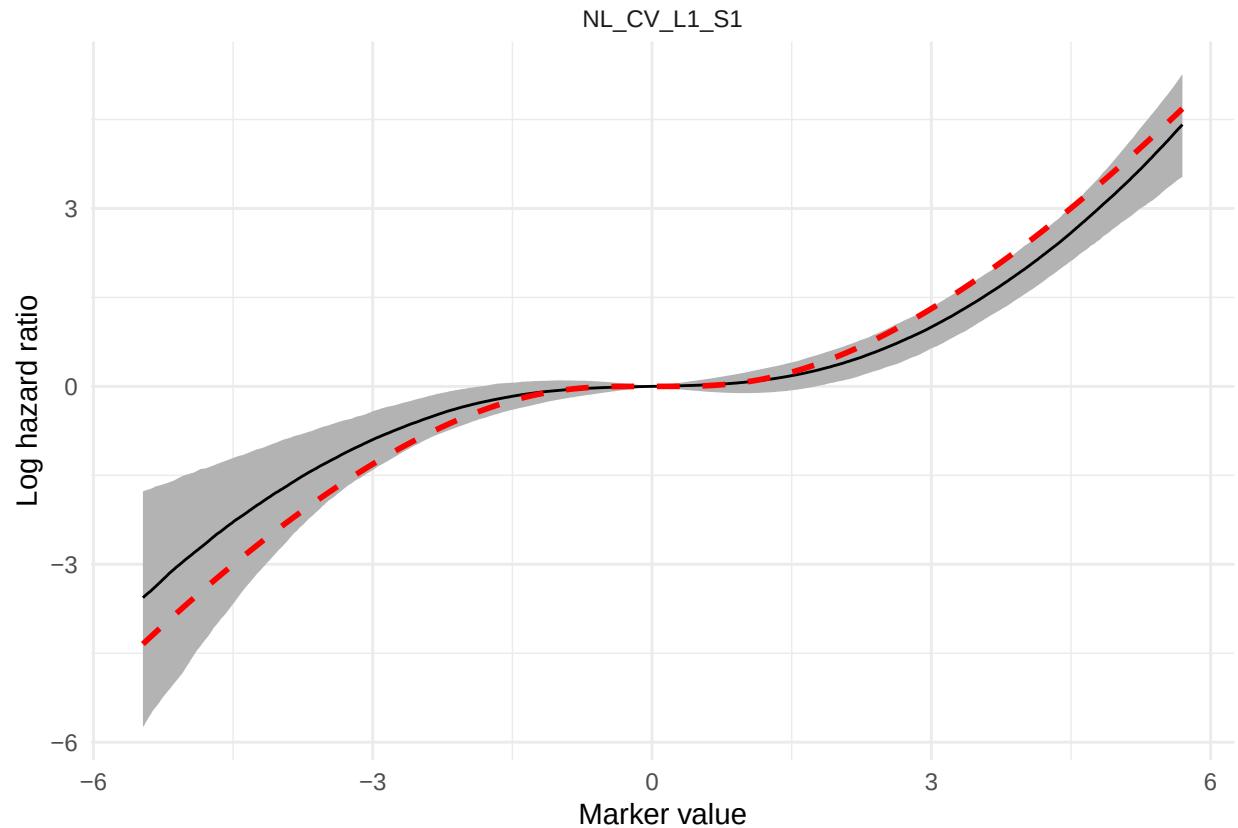
With quadratic:

```
p_lhr <- plot(M2)$NL_Associations
grid <- p_lhr$data$x
p_lhr +
  geom_line(data = data.frame(x = grid, y = F_assoc(grid)),
            aes(x = x, y = y),
            colour = "red", linetype = "dashed", linewidth = 1) +
  labs(x = "Marker value", y = "Log hazard ratio")
```



Now the spline:

```
p_lhr <- plot(M3)$NL_Associations
grid <- p_lhr$data$x
p_lhr +
  geom_line(data = data.frame(x = grid, y = F_assoc(grid)),
            aes(x = x, y = y),
            colour = "red", linetype = "dashed", linewidth = 1) +
  labs(x = "Marker value", y = "Log hazard ratio")
```



Below the center the marker protects, above it the marker harms.

The per-unit effect is not constant. It grows as the patient moves toward the edges.

The curve is drawn where the model has fitted values. The tails carry little information, so read them with care.

These structures are validated by simulation. Flexibility needs data!

Check my poster for more details: 28-P5-17

Summary

- All the models were fitted with `joint()`.
- Single formulas became lists. A single family became a vector. A single association became a vector, then a grid.

The association depends on the question we want to answer:

- Shared random effects. Individual deviation drives the risk.
- Current value. Today's level drives risk.
- Current slope. The trend drives risk.
- Cumulative. The history / accumulated exposure drives risk.
- Non-linear. The effect changes with the level.

Other extensions follow similarly, such as lagged effects or specific windows of exposure.